

SMART SCAN STRATEGY

Requirements compliance assessment and implementation roadmap

VERDICT: The application is a strong simulation and ML scheduling prototype that addresses most core software requirements. It is not yet a field-ready Electronic Support receiver scheduler because it has no live RF/SDR integration, operational data-validation pipeline, or sufficient reproducible test evidence.

Scope and evidence reviewed

- Source and design documents in EW_SIH-/: README, TECHNICAL_REPORT, METHODOLOGIES, IMPLEMENTATION_PLAN, reports, Python engine, FastAPI service, dashboard, saved models, and test file.
- This is a source-and-artifact audit, not an independent RF trial or safety certification.
- Repository evidence shows saved benchmark results generated 03 Oct 2026. The current reports use only two unseen random scenarios, despite older documentation claiming 30; this inconsistency is a validation gap.

Status key

Implemented = present in source and supported by recorded artifacts. Partial = present in a simulation/prototype form but incomplete for the stated outcome. Missing = not evidenced in the application.

1. Requirement-by-requirement assessment

Requirement	Status	Audit evidence / gap
Wide-spectrum scan with narrower instantaneous bandwidth	Implemented	ReceiverConfig models 2-18 GHz as 16 bands with 1 GHz instantaneous bandwidth and 10 ms dwell; schedulers choose one band per dwell.
No reliable prior emitter intelligence	Partial	Randomized scenario generation and online hit/miss learning reduce dependence on a fixed EOB. However, models are trained/evaluated only on synthetic, known-generator distributions; there is no live unknown-signal ingestion or drift handling.
Truth per band and time slot	Implemented	RFEnvironment precomputes activity, strongest emitter, received power, Pd, and deterministic detection outcomes for every band/slot.
Spatially scanning and frequency-agile emitters	Implemented	ScanningRadar, TrackingRadar, AgileRadar, communications emitters and beacon classes are modeled; scanning visibility is represented as temporal beam illumination.
Receiver system model	Implemented	Noise floor, free-space received power, SNR, energy detector threshold, Pd, Pfa, sensitivity, dwell, and bands are implemented.
Figures of merit	Implemented	Metrics compute Pd, Pfa, sensitivity, hit/intercept rates, raw/threat-weighted interception ratio, discovery, information age, reward, prediction accuracy, and intercept-time error.
ML scheduler minimizing intercept time and maximizing interception	Implemented	A GRU predictor scheduler and Double-DQN scheduler exist alongside model-based and open-loop baselines. DQN includes a coverage safety shield.
Training from hits and misses	Partial	Online observations feed tracker/predictor features and DQN rewards. Offline training labels and signal outcomes still originate from simulator truth; no labelled/unlabelled real receiver data workflow is supplied.
Predict interception time and interception ratio	Implemented	Exact phase-lattice periodic model and agile analytic model predict intercept time, interception ratio and lock-out; validation report compares them to simulation.
Optimal periodic scan interception	Implemented	gcd lock-out analysis, optimal_revisit / robust_revisit, and acquisition/tracking phases in ModelBasedScheduler are implemented.
Deliverable scheduler software	Implemented	Python package, trained model artifacts, FastAPI endpoints, browser dashboard, trace/reconciliation UI and demo assets are present.

Important interpretation

The system meets the expected solution as a research/simulation software package. It does not demonstrate deployment on an actual ES receiver, nor prove performance against classified or measured RF data. Those are material requirements for operational acceptance, even if they were not explicit in the original problem statement.

2. What has been implemented

Area	Implemented evidence
Simulation foundation	16-band RF environment; signal power/noise/detection model; temporal ground truth; comparable pre-drawn detection outcomes across schedulers.
Threat behavior	Rotating/scanning radar, continuous tracking radar, cyclic/random frequency agility, bursty communications, beacons, pop-up start times, threat weights and randomized laydowns.
Adaptive decision layer	Round-robin/random baselines, Thompson bandit, interpretable model-based acquisition/tracking scheduler, GRU activity predictor, and Double-DQN with replay/target network.
Periodic-scan reasoning	Exact phase-lattice analysis identifies harmonic lock-out; revisit-period optimization supplies a principled alternative to a fixed sweep.
Evaluation and presentation	Benchmark JSON/plots, theory-validation report, API endpoints, live trace/reconciliation dashboard and offline demo trace.

Saved benchmark evidence

The latest saved results (n=2 unseen random scenarios) report GRU intercept ratio 0.437 and DQN 0.380, versus round-robin 0.264. They also report theory-vs-simulation mean absolute intercept-time error of 8.7 slots (12.8%) and interception-ratio error of 0.007. Treat these as preliminary: n=2 is not enough for confidence bounds or a robust operational claim.

The README and technical report cite different, more favorable 30-scenario figures. The report must be regenerated and versioned before using any performance headline in a proposal or demonstration.

3. What still needs to be implemented

Requirement	Status	Audit evidence / gap
Live RF acquisition interface	Missing	Add SDR/receiver adapter for IQ or detector observations, timestamp discipline, tuning-control command interface, calibration, and safe fallback to deterministic sweep.
Real signal processing chain	Missing	Add channelization/energy detection from sampled IQ, pulse/burst extraction, emitter deinterleaving/classification, confidence estimates, and handling of overlapping/co-channel signals.
Spatial measurement model	Partial	Current 'spatial scan' is a time-visibility model. Add angle-of-arrival/DF, platform motion, antenna pattern, geometry, multipath/terrain and track association if spatial claims are required.
Operationally robust learning	Partial	Implement online-safe adaptation, uncertainty/OOD detection, model versioning, drift alarms, conservative policy fallback, and adversarial/jamming robustness.
Verification and reproducibility	Partial	Provide locked dependency environment, automated tests/CI, deterministic seeds, dataset manifests, run scripts, confidence intervals and ablation studies. Current test execution was blocked here because the supplied runtime lacks pytest/httpx.
Benchmark integrity	Missing	Regenerate one authoritative report with a declared train/test split, >=30-100 held-out seeds, mean/standard deviation/CI, and no conflicting numbers across README, report and dashboard.
Hardware-in-the-loop acceptance	Missing	Test tune latency, dwell timing, false-alarm performance, sensitivity, CPU timing, dropped frames and recovery using SDR/FPGA or receiver-in-loop equipment.
Multi-receiver / multi-IBW scale	Missing	Generalize scheduler action space from one receiver/one band to several receivers, different IBWs, cooperative coverage and resource constraints.

Priority distinction

Items above are not cosmetic enhancements. Live observation integration, signal processing, validation, and hardware-in-the-loop testing are the minimum bridge between a credible simulator and a deployable ES scheduling capability.

4. Recommended implementation roadmap

Phase	Indicative effort	Work	Exit criterion
0 - Baseline control	1-2 weeks	Freeze code/model versions; create one reproducible environment; repair/install test dependencies; run and archive unit/API tests; regenerate a single benchmark report with 30+ held-out seeds and CIs.	A trustworthy baseline and evidence pack.
1 - Receiver data contract	2-4 weeks	Define an adapter API for IQ/detector data, tune commands, timestamps, calibration metadata and truth/annotation capture. Replay recorded data through the current scheduler before controlling hardware.	Offline replay against real recordings.
2 - RF processing and tracking	4-8 weeks	Implement detection/channelization, pulse/burst descriptors, interference flags, emitter clustering/deinterleaving, track confidence and unknown-class handling.	Reliable observations replacing simulator truth.
3 - Safe adaptive scheduler	3-6 weeks	Add uncertainty-aware policy selection, calibrated reward/threat policy, online adaptation guardrails, drift/OOD detection, audit logs and fixed-sweep fallback.	Safe closed-loop recommendations.
4 - HIL and field validation	6-12 weeks	Connect SDR/FPGA; characterize tuning/detection latencies; evaluate across fading, jamming, pop-up and agile scenarios; quantify Pd/Pfa and intercept gains with confidence intervals.	Evidence for operational transition.

Implementation order

Do Phase 0 before asserting performance claims. Complete the replay-based integration in Phase 1 before letting ML choose live tuner actions. Introduce control authority only after Phase 2 and Phase 3 safety gates are passed.

5. Final answer

Yes - your application fulfills the main academic/software aspects of the stated problem: it simulates the scanning environment, has a receiver model and required metrics, models scanning/agile emitters, learns from hit/miss observations, supplies GRU and DQN schedulers, and includes an optimal periodic-intercept approach.

But it fulfills them as a validated-in-simulation prototype, not as a fully operational Electronic Support receiver scheduler. The decisive remaining work is integration with real receiver/SDR data, a production signal-processing and emitter-tracking chain, safety/uncertainty controls, rigorous reproducible benchmark evidence, and HIL testing.

Immediate next three actions

- Regenerate and reconcile all benchmark reports from one command and one held-out evaluation protocol; remove the conflicting $n=2$ versus $n=30$ claims.
- Define and implement a replay-first SDR/receiver adapter so the existing scheduler sees real timestamped observations without yet controlling hardware.
- Build a small HIL test campaign with timing, sensitivity, Pd/Pfa, jitter, agile-hop and periodic-scan acceptance criteria.

Audit limitations

The code was inspected and the report artifacts were reviewed. Automated tests were not run successfully in this environment because the supplied Python runtime has no pytest or httpx. No live transmitter, SDR, receiver hardware or measured RF data was available to inspect.